SMART RESTAURANT MANAGEMENT SYSTEM

Project Overview

Vision, scope, journey and delivery plan

DOCUMENT ID	SRMS-01
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Explains what we are building, why, for whom, and how the project is phased. Start here.

1. Introduction

The **Smart Restaurant Management System** lets guests scan a QR code on their table, browse the menu, place orders and pay from their own phone, while giving the restaurant manager a **live view of the whole floor**: which tables are empty, which need cleaning, and which need attention right now.

The idea was derived from a short video on AI-assisted restaurant floor management, which shows camera-based table status tracking (free / occupied, guest counts, dwell timers, wait times and a live floor map). This project delivers the same outcome in two phases: first an **event-driven system** (QR, orders, payments and staff actions drive table status), then an **AI camera detection layer** that feeds the same status engine.

2. Problem Statement

Problem today	Impact
Managers cannot see table status without walking the floor	Slow seating, idle empty tables, unhappy waiting guests
Guests wait for a waiter to take orders, bring the bill and process payment	Longer table time, lower turnover, lost tips and sales
Cleaning is triggered by chance, not by a signal	Dirty tables stay unused; inconsistent hygiene
No alert when a table is ignored or an order is delayed	Poor service recovery, bad reviews
Order errors from verbal/paper tickets	Wastage, refunds, kitchen confusion
No data on turnover, dwell time or peak hours	Guess-based staffing and decisions

3. Vision and Objectives

Vision: every table in the restaurant is self-service for the guest and fully visible to the manager in real time.

- Let guests order and pay from a table QR code without installing an app.
- Give the manager a real-time floor map with four clear states: Empty, Occupied, Needs Cleaning, Needs Attention.
- Route orders instantly to the kitchen and track each item to the table.
- Create automatic tasks for waiters and cleaners and alert on delays.
- Support UPI, cards and cash with correct GST billing.
- Provide reports on turnover, dwell time, peak hours and best-selling items.
- Prepare the architecture so AI camera detection plugs in later without redesign.

4. Scope

In scope: Phase 1 (event-driven)	In scope: Phase 2 (AI camera)	Out of scope (for now)
QR table sessions Digital menu and cart Order placement and kitchen display Online payments and cash recording Live floor map and table state engine Alerts: call waiter, delays, bill request Cleaning task queue Staff roles and admin panel Basic reports	Camera zones mapped to tables Person and object detection Auto detect: seated, left, table cleared Dwell and wait-time estimates Entrance queue counting Confidence-based suggestions to staff Privacy controls (no face storage)	Inventory and procurement Accounting software integrations Delivery aggregator integration Loyalty program Multi-country tax rules Native mobile apps (PWA first)

5. Stakeholders

Stakeholder	Interest
Restaurant owner	Revenue growth, multi-outlet visibility, cost control

Stakeholder	Interest
Manager	Live floor control, alerts, staff coordination, reports
Waiters	Clear tasks, fewer interruptions, less order taking
Kitchen staff	Accurate, prioritised, timed tickets
Cleaning staff	Know exactly which table to clean and when
Guests	Fast, contactless ordering and payment; less waiting

6. Customer Journey (End to End)

1. Guest sits at a table and scans the QR code. The code carries a secure table token.
2. The system opens (or joins) a **table session** and marks the table **Occupied**. The manager sees it turn red.
3. Guest browses the menu, customises items and adds them to the cart. Several guests at one table share the same cart.
4. Guest places the order. The kitchen display receives it. Waiter is notified for confirmation if configured.
5. Item statuses move Accepted, Preparing, Ready, Served. The guest sees live progress.
6. Guest can tap **Call Waiter** or **Request Bill**. This raises a **Needs Attention** alert on the manager map.
7. Guest pays by UPI/card, or chooses cash. Payment success closes the session.
8. Table moves to **Needs Cleaning**. A cleaning task is assigned. When the cleaner taps Done, the table becomes **Empty**.

7. What the Video Taught Us

Observed in video	Our Phase 1 equivalent	Phase 2 (AI) equivalent
Green = free table, red = occupied	Status from QR scan, order and payment events	Person detected in table zone
Table label with guest count and timer	Guest count from session; timer from session start	Count people in zone
Occupancy panel (tables, guests, capacity %)	Calculated from live table states	Same panel, AI-verified
Wipe-down turns table to FREE	Cleaner taps Done	Detect cleaner activity and clear table
Wait time and entrance queue	Waitlist with manual add; wait timer	Count people at entrance
Long-stay timer (e.g. 1 hour 15 min)	Session duration alerts	Dwell-time analytics
Detect customer and staff need	Call waiter, order delay, inactivity alerts	Detect raised hand or ignored table

8. Delivery Phases

Phase	Goal	Outcome
Phase 1	Event-driven core	Working QR ordering, payments, manager floor map, alerts, cleaning tasks
Phase 2	AI camera detection	Cameras confirm or auto-update table status; queue and dwell analytics
Phase 3	Scale and growth	Multi-outlet, reservations, loyalty, deeper analytics

9. Success Metrics (KPIs)

KPI	Target (first 3 months of use)
Share of orders placed via QR	At least 70%

KPI	Target (first 3 months of use)
Average table turnover time	Reduce by 15%
Time from guest leaving to table ready	Under 5 minutes
Alert acknowledge time (call waiter)	Under 90 seconds
Order accuracy (no manual re-entry)	At least 98%
Payment success rate (online)	At least 97%
Floor map accuracy vs reality (Phase 1)	At least 95%

10. Assumptions and Constraints

- Guests have a smartphone with a camera and mobile data or restaurant Wi-Fi.
- The restaurant has reliable internet; kitchen and manager screens are tablets, PCs or phones.
- Primary market is India: INR currency, UPI payments, GST invoicing, Indian data-protection norms.
- Staff will tap status buttons (cleaned, served). Phase 2 reduces this effort but does not remove manual override.
- Phase 1 is a responsive web app (PWA). Native apps are optional later.

11. Glossary

Term	Meaning
Table session	The period from first scan or seating until payment is completed and the table is released
Needs Attention	An overlay flag raised when a table has an open alert such as a waiter call or delayed order
KDS	Kitchen Display System, the screen where the kitchen sees tickets
PWA	Progressive Web App; a website that behaves like an app, no store install needed
Dwell time	How long guests occupy a table
State engine	The backend component that is the single source of truth for table status

SMART RESTAURANT MANAGEMENT SYSTEM

User Roles and Permissions

Who uses the system and what each role may do

DOCUMENT ID	SRMS-02
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Defines every actor, their goals, permissions, restrictions and authentication method.

1. Overview

The system has six human roles and two automated actors. Access is controlled by role-based access control (RBAC). Every staff action is tied to a user and recorded in the audit log.

Role	Who they are	Login method	Primary device
Customer (Guest)	Diner at a table	None required: QR session. Optional phone OTP for receipts	Own phone
Waiter / Server	Floor staff	Staff ID + PIN	Phone or tablet
Kitchen Staff	Chef, cooks	Staff ID + PIN	Kitchen tablet / screen
Cleaner / Busser	Table cleaning staff	Staff ID + PIN	Phone
Manager	Shift or floor manager	Email + password (optional 2FA)	Tablet / PC / phone
Owner / Admin	Owner or system administrator	Email + password + 2FA	PC
System (Rules Engine)	Timers and scheduled jobs	Internal service account	Backend
AI Vision Service (Phase 2)	Camera detection service	Signed service token	Edge or cloud server

2. Role Profiles

2.1 Customer (Guest)

Can do

- Open the menu from the table QR; search and filter (veg, category, price).
- Customise items; add to the shared table cart; place orders (multiple rounds allowed).
- See live order status per item.
- Call waiter, request bill, report an issue.
- Pay online or choose cash; split the bill (Phase 1.5); leave feedback.

Restrictions

- Cannot see other tables or other sessions.
- Cannot cancel items once the kitchen has accepted them (must ask staff).
- Session expires after payment or inactivity timeout.

2.2 Waiter / Server

Can do

- See assigned tables and their live state.
- Acknowledge and resolve alerts (call waiter, bill request).
- Place or edit an order on behalf of a guest; seat a walk-in manually.
- Mark items as Served.
- Record cash payments; request manager approval for discounts/cancellations.

Restrictions

- Cannot change menu or prices.
- Cannot issue refunds or view revenue reports.
- Sees only their section unless the manager grants more.

2.3 Kitchen Staff

Can do

- See incoming order tickets in time order with table number and notes.

- Accept, start preparing, mark Ready.
- Mark an item out of stock (temporarily unavailable).

Restrictions

- Cannot see payment data.
- Cannot cancel a paid order.
- Cannot edit prices.

2.4 Cleaner / Busser

Can do

- See the cleaning queue: tables in Needs Cleaning state, oldest first.
- Start a cleaning task and mark it Done.
- Flag a table as Out of Service (damage, spill) with a note.

Restrictions

- Cannot view orders or payments.
- Cannot change any state except cleaning related ones.

2.5 Manager

Can do

- Live floor map with all tables, alerts, KPIs and timers.
- Override any table state with a reason (logged).
- Assign tables to waiters and tasks to cleaners.
- Approve discounts, cancellations and refunds up to a limit.
- Manage menu availability, staff shifts and view shift reports.
- Manage reservations and the waiting list.

Restrictions

- Cannot delete audit logs.
- Cannot change system-level or payment gateway settings (Owner only).

2.6 Owner / Admin

Can do

- Everything a manager can do, plus: create outlets, floors, tables and QR codes.
- Manage staff accounts and roles.
- Configure taxes, service charge, payment gateway keys, alert thresholds.
- Full reports, exports and audit log access.
- Configure camera integration (Phase 2).

Restrictions

- Actions on payment settings and refunds above limit require 2FA.

3. Permission Matrix

Y = allowed, - = not allowed, Own = only their own records or assigned tables, Appr = needs approval.

Capability	Guest	Waiter	Kitchen	Cleaner	Manager	Owner
Open menu / place order via QR	Y	-	-	-	-	-
Place / edit order for guest	-	Y	-	-	Y	Y
View kitchen tickets	-	Y	Y	-	Y	Y

Capability	Guest	Waiter	Kitchen	Cleaner	Manager	Owner
Update item status (accept, ready)	-	Served only	Y	-	Y	Y
View live floor map	-	Own section	-	-	Y	Y
Override table status	-	Limited	-	Cleaning only	Y	Y
Raise / view alerts	Call only	Y	-	-	Y	Y
Resolve alerts	-	Y	-	-	Y	Y
Cleaning tasks (start, done)	-	-	-	Y	Y	Y
Pay bill / record cash	Pay own bill	Cash	-	-	Y	Y
Discount / item cancel	-	Appr	-	-	Y	Y
Refund	-	-	-	-	Up to limit	Y
Edit menu and prices	-	-	Availability only	-	Y	Y
Manage tables and QR codes	-	-	-	-	-	Y
Manage staff and roles	-	-	-	-	Shifts only	Y
Reports	-	-	-	-	Y	Y (all outlets)
System / gateway settings	-	-	-	-	-	Y
View audit log	-	-	-	-	Read	Y

4. Authentication and Session Rules

- **Guests:** a short-lived session token is issued when the QR is scanned. It is bound to one table session and expires on payment, after the configured idle time, or when the manager closes the table.
- **Staff:** PIN login for floor roles (fast shift start); email and password for manager and owner. Failed attempts are rate limited and locked after repeated failure.
- **Tokens:** short-lived access token plus refresh token. Role and restaurant ID are embedded and verified on every request.
- **Multi-outlet:** a user belongs to one restaurant (outlet); owners can belong to several.
- **Audit:** logins, overrides, discounts, refunds, menu and settings changes are recorded with user, time and device.

5. Automated Actors

Actor	What it does	Allowed actions
System rules engine	Runs timers: idle session, order delay, long stay, unattended table	Raise alerts, create cleaning tasks, expire sessions
AI vision service (Phase 2)	Sends detection events (person seated, table cleared, queue size)	Create suggestions or set status per configured confidence rules; never edit payments

SMART RESTAURANT MANAGEMENT SYSTEM

Core Modules

Functional modules, features and dependencies

DOCUMENT ID	SRMS-03
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Breaks the system into modules with features, inputs, outputs and dependencies.

1. Module Map

The system is divided into functional modules. Phase 1 modules are the MVP; Module M14 is added in Phase 2.

ID	Module	Purpose	Phase
M1	QR and Table Session	Identify the table, open and manage the session	1
M2	Digital Menu	Present items, variants, modifiers, availability	1
M3	Cart and Ordering	Shared cart, place order, rounds, order history	1
M4	Kitchen Display (KDS)	Tickets, preparation status, timing	1
M5	Billing and Payments	Bill, tax, UPI/card/cash, refunds	1
M6	Table Management and Live Floor Map	Table states, floor map, manual override	1
M7	Alerts and Attention Engine	Raise, route and resolve table alerts	1
M8	Cleaning and Task Management	Cleaning queue, staff tasks	1
M9	Authentication and Staff	Users, roles, shifts, PIN login	1
M10	Admin and Configuration	Menu, tables, QR codes, taxes, thresholds	1
M11	Real-time Notifications	WebSocket events, push, sound	1
M12	Reports and Analytics	Sales, turnover, dwell, peak hours	1 basic, 3 advanced
M13	Feedback and Reservations/Waitlist	Ratings, comments, waiting list, bookings	1.5
M14	AI Camera Vision	Detect seating, clearing, queues and attention needs	2

M1. QR and Table Session

Entry point for every guest interaction.

- Each table has a unique QR encoding a random **table token** (not the table number alone, to prevent guessing).
- Scan opens the menu in the browser and creates or joins the active session for that table.
- If a session is active, new scanners join it (shared cart). If it has been closed, a new one starts.
- Optional guest count and name prompt.
- Session expiry on payment, idle timeout or manager closure.
- Admin can regenerate or print QR codes (PDF sheet with table labels).

Input: Table token **Output:** Session ID, table context

M2. Digital Menu

Everything the guest sees before ordering.

- Categories, items with photo, description, price, veg/non-veg marker, tags, prep time.
- Variants (half/full), modifier groups (extra cheese, spice level) with min/max rules.
- Search, filters, popular and recommended sections.
- Real-time availability: sold-out items are greyed out instantly.
- Multi-language support (English, Hindi, Gujarati) in a later release.

Input: Menu data **Output:** Menu JSON for UI

M3. Cart and Ordering

Turns browsing into a validated order.

- Shared cart per session with live sync between guests at the same table.
- Special instructions per item and per order.
- Price, tax and total are calculated on the server, never trusted from the client.

- Multiple order rounds in one session; each round is a separate order under one bill.
- Place order triggers kitchen ticket and waiter notification; optional waiter confirmation mode.
- Cancellation rule: allowed until kitchen accepts; later needs staff approval.

Input: Cart, session **Output:** Order, order items

M4. Kitchen Display (KDS)

Accurate tickets for the kitchen.

- Tickets appear in real time, oldest first, with table, items, modifiers and notes.
- Item status: Pending, Accepted, Preparing, Ready.
- Colour timers turn amber and red when an item exceeds its target prep time.
- Mark item out of stock; menu updates immediately.
- Optional station routing (grill, bar, dessert) in a later release.

Input: Orders **Output:** Item status events

M5. Billing and Payments

Fair, correct and fast payment.

- One running bill per session combining all orders.
- GST breakdown, service charge toggle, discounts (manager approved), tips.
- Online payment via UPI intent/QR, cards and wallets through a payment gateway (Razorpay or equivalent).
- Cash: waiter records receipt; bill closes.
- Idempotent payment handling and webhook verification to avoid double charge or missed payment.
- Partial payment and split bill (equal split first, item split later).
- Refunds with reason and approval; PDF/WhatsApp-ready receipt.

Input: Bill **Output:** Payment, receipt

M6. Table Management and Live Floor Map

The manager's main screen. Single source of truth for table status.

- Table states: **Available, Reserved, Occupied, Payment Pending, Needs Cleaning, Cleaning In Progress, Out of Service**.
- **Needs Attention** is an overlay flag from open alerts, not a separate state.
- Drag-and-drop floor layout editor in admin; multiple floors or zones.
- Each table tile shows label, state colour, guest count, session timer and alert icon.
- Side panel: orders, bill status, alerts and action buttons (seat, clean, override, close).
- Header KPIs: tables occupied, guests seated, capacity %, average dwell time, active alerts.
- Every state change is logged with actor and reason.

Input: Events from M1, M3, M5, M7, M8 **Output:** Table status, live map

M7. Alerts and Attention Engine

Makes sure no table is ignored.

- Alert types: Call Waiter, Request Bill, Order Delayed, Payment Failed, Long Wait (no order after seating), Unattended Table, Long Stay, Help.
- Rules and thresholds are configurable per restaurant (for example no order within 10 minutes of seating).
- Alert lifecycle: Open, Acknowledged (by whom), Resolved. Unacknowledged alerts escalate to the manager.
- Routing to the assigned waiter first, manager as fallback.
- Visual and sound cues on the floor map and staff phones.

Input: Events and timers **Output:** Alerts, escalations

M8. Cleaning and Task Management

Closes the loop from guest leaving to table ready.

- When payment completes (or the manager closes the table), a cleaning task is created and the table becomes Needs Cleaning.
- Cleaners see a prioritised queue (oldest first, reserved tables boosted).
- Start and Done actions with timestamps; average clean time tracked.
- Optional checklist (wipe, reset cutlery, sanitise).
- Manager can assign a task to a specific cleaner or let it be first-come.

Input: Table state **Output:** Cleaning tasks, table released

M9. Authentication and Staff

Secure identity for staff and sessions.

- Staff CRUD, role assignment, active/inactive.
- PIN login for floor staff; email/password plus optional 2FA for manager/owner.
- Shifts and table assignment per waiter.
- Audit trail of sensitive actions.

Input: Credentials **Output:** Tokens, permissions

M10. Admin and Configuration

Everything the owner configures once and tweaks later.

- Menu management with bulk import (CSV), images, availability schedule.
- Floors, tables, capacity, layout and QR generation.
- Tax rates, service charge, currency, receipt header.
- Alert thresholds and session timeouts.
- Payment gateway keys (owner only, stored encrypted).

Input: Admin input **Output:** Configuration

M11. Real-time Notifications

Live updates everywhere.

- WebSocket channels per restaurant (and per table for guests) for table, order, alert and payment events.
- Browser push and in-app sound for staff devices.
- Graceful fallback to polling if the socket drops; automatic resync on reconnect.

Input: Domain events **Output:** Live UI updates

M12. Reports and Analytics

Turns operations into decisions.

- Sales by day, hour, category, item; payment method split.
- Table turnover, average dwell time, average cleaning time.
- Alert response time, order preparation time.
- Peak-hour heat map by table and time.
- Export to CSV/PDF.

Input: All data **Output:** Dashboards, exports

M13. Feedback, Reservations and Waitlist

Experience and demand management.

- Post-payment rating and comment; low ratings alert the manager.
- Waitlist: add party, estimate wait, notify by SMS/WhatsApp when a table frees.

- Reservations: hold a table for a time window; the table shows Reserved on the map.

Input: Guest input **Output:** Feedback, queue

M14. AI Camera Vision (Phase 2)

Automatic table awareness from CCTV, as demonstrated in the video.

- Map each camera view to table zones; detect people, chairs and tables with an object detection model.
- Infer: guests seated, guests left, table cleared, staff near table, long unattended table, queue at entrance.
- Send events to the same state engine with a confidence score. High confidence updates automatically; low confidence becomes a suggestion for staff to confirm.
- Processing at the edge where possible; no face recognition or identity storage; store only counts, states and short event snapshots if enabled.
- Manual override always wins and feeds back to improve thresholds.

Input: Video streams **Output:** Vision events

2. Module Dependencies

Module	Depends on	Used by
M1 QR/Session	M10 (tables and QR)	M2, M3, M5, M6
M3 Ordering	M1, M2	M4, M5, M6
M5 Billing	M3	M6, M8, M12
M6 Floor map	M1, M3, M5, M7, M8	Manager UI, M12
M7 Alerts	M1, M3, M5, timers	M6, M11
M8 Cleaning	M5, M6	M6, M12
M14 AI vision	M6, M7, M10	M6, M7, M12

SMART RESTAURANT MANAGEMENT SYSTEM

Database Schema

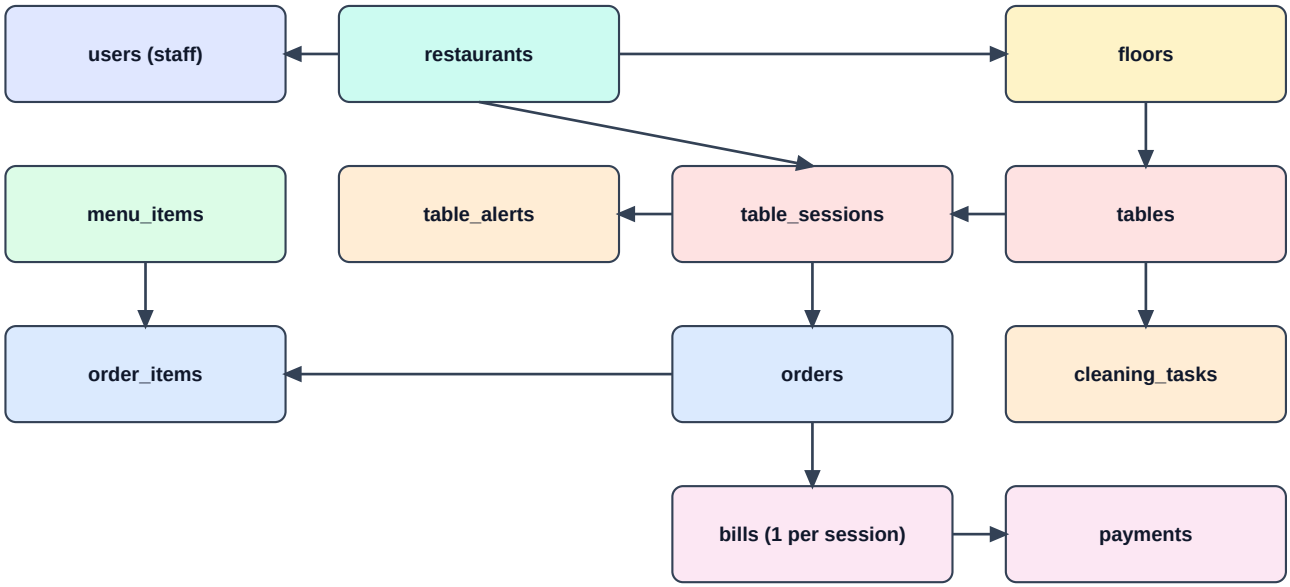
PostgreSQL tables, relations, enums and indexes

DOCUMENT ID	SRMS-04
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Complete data model for Phase 1 and the Phase 2 AI camera extension.

1. Design Principles

- **Database:** PostgreSQL 15+. Primary keys are UUIDs; timestamps are `timestamptz` in UTC.
- **Multi-tenant ready:** every business table carries `restaurant_id` so multiple outlets can share one database.
- **Snapshots:** order items store name and price at order time so later menu edits never change history.
- **Money:** `numeric(12,2)`, never floating point. Currency is INR by default.
- **Soft delete:** menu and staff records use `is_active` or `deleted_at`; financial records are never deleted.
- **Audit:** status changes and sensitive actions are written to log tables.
- **Single source of truth:** table status lives in `tables.status` and is changed only by the state engine, with every change logged.

2. Entity Relationship Summary



Reading the diagram: a restaurant has floors and staff; a floor has tables; a table has many sessions over time; each session has orders (with items from the menu), one bill, and the bill has payments. Alerts and cleaning tasks also hang off the table and session.

3. Enumerations

Enum	Values
user_role	OWNER, MANAGER, WAITER, KITCHEN, CLEANER
table_status	AVAILABLE, RESERVED, OCCUPIED, PAYMENT_PENDING, NEEDS_CLEANING, CLEANING_IN_PROGRESS, OUT_OF_SERVICE
session_status	ACTIVE, CLOSED, EXPIRED
order_status	PLACED, CONFIRMED, IN_PROGRESS, READY, SERVED, CANCELLED
item_status	PENDING, ACCEPTED, PREPARING, READY, SERVED, CANCELLED
bill_status	OPEN, PAID, PARTIALLY_PAID, VOID
payment_method	UPI, CARD, WALLET, CASH
payment_status	CREATED, PENDING, SUCCESS, FAILED, REFUNDED, PARTIALLY_REFUNDED

Enum	Values
alert_type	CALL_WAITER, REQUEST_BILL, ORDER_DELAYED, PAYMENT_FAILED, LONG_WAIT, UNATTENDED, LONG_STAY, HELP, AI_SUGGESTION
alert_status	OPEN, ACKNOWLEDGED, RESOLVED
task_status	PENDING, IN_PROGRESS, DONE, CANCELLED
actor_type	CUSTOMER, STAFF, SYSTEM, AI

4. Table Definitions

4.1 Organisation and Staff

restaurants

One row per outlet.

Column	Type	Constraints / Notes
id	uuid PK	default gen_random_uuid()
name	varchar(150)	not null
slug	varchar(80)	unique; used in URLs
address, phone	text, varchar(20)	
gstin	varchar(15)	nullable
currency	char(3)	default INR
timezone	varchar(50)	default Asia/Kolkata
settings	jsonb	taxes, service charge, thresholds, session timeouts
created_at	timestamptz	default now()

users

Staff accounts (guests are not users).

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	references restaurants(id)
name	varchar(120)	not null
email	varchar(160)	unique per restaurant; nullable for PIN-only staff
phone	varchar(20)	
password_hash	text	argon2/bcrypt; nullable
pin_hash	text	hashed staff PIN
role	user_role	not null
is_active	boolean	default true
last_login_at	timestamptz	
created_at	timestamptz	

staff_shifts

Who is on duty and which tables they cover.

Column	Type	Constraints / Notes
id	uuid PK	
user_id	uuid FK	references users(id)
floor_id	uuid FK	nullable
assigned_table_ids	uuid[]	tables this waiter covers
starts_at, ends_at	timestamptz	

4.2 Floor and Tables

floors

Zones or floors (Indoor, Terrace, Family section).

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
name	varchar(80)	
sort_order	int	default 0

tables

Physical tables and their live status.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
floor_id	uuid FK	
label	varchar(20)	e.g. T04; unique per restaurant
capacity	smallint	seats
shape	varchar(12)	round, square, rect
pos_x, pos_y, width, height	numeric	floor-map layout
status	table_status	default AVAILABLE; changed only by state engine
status_changed_at	timestamptz	drives timers
current_session_id	uuid FK null	active session if any
qr_token	varchar(64)	unique, random, rotatable
assigned_waiter_id	uuid FK null	default waiter
is_active	boolean	default true

4.3 Sessions

table_sessions

One occupancy period of a table.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
table_id	uuid FK	indexed
status	session_status	default ACTIVE
opened_by	actor_type	CUSTOMER (QR), STAFF (seated), AI
guest_count	smallint	nullable
guest_phone	varchar(20)	optional, for receipt
started_at	timestamptz	not null
ended_at	timestamptz	
last_activity_at	timestamptz	for idle expiry
close_reason	varchar(40)	PAID, TIMEOUT, MANAGER, WALK_OUT

4.4 Menu

menu_categories

Menu sections.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
name	varchar(100)	
sort_order	int	
is_active	boolean	

menu_items

Dishes and drinks.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
category_id	uuid FK	
name	varchar(150)	
description	text	
price	numeric(12,2)	base price
tax_rate	numeric(5,2)	GST percent
image_url	text	
is_veg	boolean	
is_available	boolean	sold-out toggle
prep_time_min	smallint	target preparation time
tags	text[]	spicy, bestseller
sort_order	int	

item_variants

Size or portion options.

Column	Type	Constraints / Notes
id	uuid PK	
item_id	uuid FK	
name	varchar(60)	Half / Full
price	numeric(12,2)	replaces base price

modifier_groups and modifiers

Customisation (extra cheese, spice level).

Column	Type	Constraints / Notes
modifier_groups.id	uuid PK	
modifier_groups.item_id	uuid FK	
modifier_groups.name	varchar(80)	
modifier_groups.min_select, max_select	smallint	selection rules
modifiers.id	uuid PK	
modifiers.group_id	uuid FK	
modifiers.name	varchar(80)	
modifiers.price_delta	numeric(12,2)	added to price

4.5 Orders

orders

One order round within a session.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
session_id	uuid FK	indexed
table_id	uuid FK	denormalised for speed
order_no	int	per-restaurant daily sequence
status	order_status	default PLACED
subtotal, tax_total, discount_total, total	numeric(12,2)	server calculated
notes	text	
placed_by_type	actor_type	CUSTOMER or STAFF
placed_by_user_id	uuid FK null	when staff placed
placed_at	timestamptz	

order_items

Line items with their own status for kitchen tracking.

Column	Type	Constraints / Notes
id	uuid PK	
order_id	uuid FK	
menu_item_id	uuid FK	
variant_id	uuid FK null	
name_snapshot	varchar(150)	name at order time
unit_price	numeric(12,2)	price at order time
quantity	smallint	at least 1
modifiers_snapshot	jsonb	chosen modifiers and deltas
line_total	numeric(12,2)	
status	item_status	default PENDING
instructions	text	
accepted_at, ready_at, served_at	timestamptz	timing analytics
cancel_reason	text	

4.6 Billing and Payments

bills

Running bill for a session.

Column	Type	Constraints / Notes
id	uuid PK	
session_id	uuid FK unique	one bill per session
subtotal, tax_total, service_charge, discount_total, tip	numeric(12,2)	
grand_total	numeric(12,2)	
amount_paid	numeric(12,2)	default 0
status	bill_status	default OPEN
invoice_no	varchar(30)	unique, sequential for GST
generated_at, paid_at	timestamptz	

payments

Every payment attempt.

Column	Type	Constraints / Notes
id	uuid PK	
bill_id	uuid FK	
method	payment_method	
provider	varchar(30)	razorpay
provider_order_id, provider_payment_id	varchar(80)	unique where not null
amount	numeric(12,2)	
status	payment_status	
idempotency_key	varchar(80)	unique; prevents duplicate charge
received_by_user_id	uuid FK null	for cash
raw_payload	jsonb	webhook body
paid_at	timestamptz	

refunds

Refund records.

Column	Type	Constraints / Notes
id	uuid PK	
payment_id	uuid FK	
amount	numeric(12,2)	
reason	text	
approved_by	uuid FK	manager/owner
status	varchar(20)	
created_at	timestamptz	

4.7 Attention, Cleaning and Logs

table_alerts

Things that need staff attention.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
table_id	uuid FK	
session_id	uuid FK null	
type	alert_type	
severity	smallint	1 info, 2 warning, 3 critical
message	text	
raised_by	actor_type	
status	alert_status	default OPEN
assigned_to	uuid FK null	waiter
raised_at, acknowledged_at, resolved_at	timestamptz	response-time metrics
escalated	boolean	default false

cleaning_tasks

Cleaning queue.

Column	Type	Constraints / Notes
id	uuid PK	
table_id	uuid FK	
session_id	uuid FK null	
status	task_status	
assigned_to	uuid FK null	cleaner
priority	smallint	raised if table is reserved soon
created_at, started_at, completed_at	timestamptz	
notes	text	

table_status_log

Immutable history of every table state change.

Column	Type	Constraints / Notes
id	bigserial PK	
table_id	uuid FK	indexed
from_status, to_status	table_status	
reason	varchar(80)	
actor_type	actor_type	
actor_id	uuid null	user id if staff
created_at	timestamptz	

audit_logs

Sensitive actions.

Column	Type	Constraints / Notes
id	bigserial PK	
restaurant_id	uuid FK	
actor_id	uuid FK null	
action	varchar(60)	DISCOUNT_APPLIED, REFUND, MENU_PRICE_CHANGED
entity, entity_id	varchar, uuid	
diff	jsonb	before and after
ip, user_agent	text	
created_at	timestamptz	

4.8 Feedback, Waitlist and Notifications (Phase 1.5)

feedback

Guest ratings.

Column	Type	Constraints / Notes
id	uuid PK	
session_id	uuid FK	
rating	smallint	1 to 5
comment	text	
created_at	timestamptz	

reservations and waitlist_entries

Demand management.

Column	Type	Constraints / Notes
reservations.id	uuid PK	
reservations.table_id	uuid FK null	
reservations.name, phone, party_size	mixed	
reservations.reserved_for	timestamptz	
reservations.status	varchar(20)	BOOKED, SEATED, NO_SHOW, CANCELLED
waitlist_entries.id	uuid PK	
waitlist_entries.name, phone, party_size	mixed	
waitlist_entries.joined_at, notified_at, seated_at	timestamptz	
waitlist_entries.status	varchar(20)	WAITING, NOTIFIED, SEATED, LEFT

4.9 Phase 2: AI Camera Tables

cameras

Registered camera sources.

Column	Type	Constraints / Notes
id	uuid PK	
restaurant_id	uuid FK	
floor_id	uuid FK	
name	varchar(80)	
stream_url_encrypted	text	RTSP URL stored encrypted
status	varchar(20)	ONLINE, OFFLINE
last_seen_at	timestamptz	

camera_table_zones

Maps a polygon in the camera image to a table.

Column	Type	Constraints / Notes
id	uuid PK	
camera_id	uuid FK	
table_id	uuid FK	
polygon	jsonb	list of normalised x,y points

vision_events

Detections sent by the AI service.

Column	Type	Constraints / Notes
id	bigserial PK	
camera_id	uuid FK	
table_id	uuid FK null	
event_type	varchar(40)	PERSON_SEATED, TABLE_CLEARED, QUEUE_COUNT, LONG_UNATTENDED
person_count	smallint	
confidence	numeric(4,3)	0 to 1
applied	boolean	true if state engine used it
snapshot_url	text null	optional, privacy controlled
detected_at	timestamptz	

5. Key Indexes and Constraints

Purpose	Definition
Find active session fast	Unique partial index on table_sessions(table_id) where status = 'ACTIVE' (guarantees one active session per table)
Floor map query	Index tables(restaurant_id, floor_id, status)
Kitchen queue	Index order_items(status, accepted_at) where status in PENDING/ACCEPTED/PREPARING
Open alerts	Index table_alerts(restaurant_id, status, raised_at) where status != 'RESOLVED'
Cleaning queue	Index cleaning_tasks(status, created_at) where status in PENDING/IN_PROGRESS
No duplicate payment	Unique on payments(idempotency_key) and payments(provider_payment_id)

Purpose	Definition
Invoice sequence	Unique bills(restaurant_id, invoice_no)
QR lookup	Unique tables(qr_token)
Reports	Index orders(restaurant_id, placed_at), payments(paid_at)

6. Data Retention and Privacy

- Guest phone numbers are optional and stored only for receipts, waitlist notifications and feedback.
- Financial records are retained as required by Indian tax rules; personal data can be anonymised on request.
- Phase 2 stores counts and states, not faces. Snapshots are off by default and auto-deleted after a short retention window.
- Daily encrypted backups with point-in-time recovery; backups tested regularly.

SMART RESTAURANT MANAGEMENT SYSTEM

Software Requirements Specification

Functional and non-functional requirements

DOCUMENT ID	SRMS-05
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Formal, numbered requirements with priorities, interfaces, use cases and acceptance criteria.

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the functional and non-functional requirements of the Smart Restaurant Management System. It is intended for the project owner, designers, developers and testers.

1.2 Scope

The system allows guests to order and pay through a table QR code and gives restaurant staff and managers real-time control of orders, tables, alerts and cleaning. Phase 1 is event-driven. Phase 2 adds AI camera detection feeding the same table state engine.

1.3 Definitions

Term	Definition
SRS	Software Requirements Specification
QR session	A temporary guest session created by scanning a table QR code
State engine	Backend component owning table status transitions
KDS	Kitchen Display System
Must / Should / Could	Priority: Must is required for MVP, Should is important, Could is optional

1.4 References

- Project Overview (SRMS-01), User Roles (SRMS-02), Core Modules (SRMS-03), Database Schema (SRMS-04).
- Source inspiration: video on AI-assisted restaurant floor management (camera-based table status, wait and dwell tracking).

2. Overall Description

2.1 Product Perspective

A web-based system with four client interfaces (guest PWA, manager dashboard, staff apps for waiter/kitchen/cleaner, admin panel) sharing one backend, database and real-time channel. It integrates with a payment gateway and, in Phase 2, with camera streams.

2.2 User Classes

Customer, Waiter, Kitchen Staff, Cleaner, Manager, Owner/Admin as defined in SRMS-02.

2.3 Operating Environment

- Guests: current mobile browsers (Chrome, Safari, Samsung Internet) on Android 8+/iOS 14+.
- Staff: Chrome or Edge on tablet, phone or PC. Kitchen display runs full screen on a tablet or monitor.
- Server: Linux, containerised, managed PostgreSQL and Redis, HTTPS everywhere.

2.4 Constraints and Assumptions

- Internet connectivity is required in the restaurant; staff apps tolerate short drops with automatic resync.
- Payments must go through a PCI-DSS compliant gateway; the system never stores card data.
- Billing must comply with Indian GST invoice requirements.
- Phase 2 camera analysis must not store or recognise faces.

3. Functional Requirements

3.1 QR and Session (M1)

ID	Requirement	Priority
FR-QR-01	Each table shall have a unique, unguessable QR token that opens the menu for that table.	Must
FR-QR-02	Scanning shall create a session if none is active, or join the active session, and mark the table Occupied.	Must
FR-QR-03	The system shall allow only one active session per table.	Must
FR-QR-04	Sessions shall expire after payment, configurable idle time, or manager closure.	Must
FR-QR-05	Admin shall generate and print QR sheets and rotate a table token.	Should
FR-QR-06	Guests may enter guest count and optional phone number.	Could

3.2 Menu (M2)

ID	Requirement	Priority
FR-MN-01	The guest shall browse categories and items with image, price, description and veg marker.	Must
FR-MN-02	The system shall support variants and modifier groups with min/max rules.	Must
FR-MN-03	Sold-out items shall be unavailable to order immediately after staff mark them.	Must
FR-MN-04	The guest shall search and filter the menu.	Should
FR-MN-05	The menu shall support multiple languages.	Could

3.3 Cart and Ordering (M3)

ID	Requirement	Priority
FR-OR-01	Guests at one table shall share one cart synchronised in real time.	Must
FR-OR-02	The server shall compute all prices, taxes and totals; client values are not trusted.	Must
FR-OR-03	A guest shall be able to place multiple order rounds in one session.	Must
FR-OR-04	Placing an order shall create kitchen tickets and notify the assigned waiter.	Must
FR-OR-05	The guest shall see per-item live status.	Must
FR-OR-06	Items may be cancelled by the guest until the kitchen accepts; afterwards by staff only.	Should
FR-OR-07	Staff shall place or edit orders on behalf of guests.	Must

3.4 Kitchen (M4)

ID	Requirement	Priority
FR-KT-01	The KDS shall display new tickets in real time ordered by time with table, items, modifiers and notes.	Must
FR-KT-02	Kitchen staff shall set item status Accepted, Preparing, Ready.	Must
FR-KT-03	The KDS shall highlight tickets exceeding target preparation time.	Should
FR-KT-04	Kitchen staff shall mark items out of stock.	Must

3.5 Billing and Payment (M5)

ID	Requirement	Priority
FR-PY-01	The system shall maintain one running bill per session aggregating all orders.	Must
FR-PY-02	The bill shall show GST breakdown, optional service charge, discounts and tip.	Must
FR-PY-03	The guest shall pay online via UPI or card through the payment gateway.	Must
FR-PY-04	Payment confirmation shall be taken from verified gateway webhooks, not only from the client.	Must

ID	Requirement	Priority
FR-PY-05	Payment requests shall be idempotent to prevent double charge.	Must
FR-PY-06	Staff shall record cash payments.	Must
FR-PY-07	Discounts and refunds shall require manager approval and be audited.	Must
FR-PY-08	The system shall generate a sequential tax invoice and shareable receipt.	Must
FR-PY-09	The system shall support equal bill split and partial payment.	Should

3.6 Table Management and Floor Map (M6)

ID	Requirement	Priority
FR-TB-01	The system shall maintain table status: Available, Reserved, Occupied, Payment Pending, Needs Cleaning, Cleaning In Progress, Out of Service.	Must
FR-TB-02	Status transitions shall be performed only by the state engine according to the defined rules (SRMS-06).	Must
FR-TB-03	The manager dashboard shall show all tables on a floor map with colour, label, guest count, timer and alert icon, updating within 2 seconds.	Must
FR-TB-04	The dashboard shall show occupancy KPIs: tables occupied, guests seated, capacity percent, average dwell time, open alerts.	Must
FR-TB-05	Managers shall override a table state with a mandatory reason, logged.	Must
FR-TB-06	Every status change shall be written to the status log with actor and time.	Must
FR-TB-07	Owner shall design floors and table layout visually.	Should
FR-TB-08	Staff shall seat walk-in guests manually.	Must

3.7 Alerts (M7)

ID	Requirement	Priority
FR-AL-01	Guests shall be able to trigger Call Waiter and Request Bill alerts.	Must
FR-AL-02	The system shall automatically raise alerts for delayed orders, long wait without ordering, unattended table, long stay and payment failure.	Must
FR-AL-03	Alert thresholds shall be configurable per restaurant.	Must
FR-AL-04	Alerts shall be routed to the assigned waiter and escalate to the manager if not acknowledged within a set time.	Must
FR-AL-05	A table with any open alert shall display Needs Attention on the floor map.	Must
FR-AL-06	Alert acknowledge and resolve times shall be recorded.	Must

3.8 Cleaning (M8)

ID	Requirement	Priority
FR-CL-01	A cleaning task shall be created automatically when a session closes.	Must
FR-CL-02	Cleaners shall see a queue ordered by priority and age.	Must
FR-CL-03	Cleaners shall start and complete tasks; completion sets the table Available.	Must
FR-CL-04	Cleaners shall mark a table Out of Service with a note.	Should
FR-CL-05	The system shall record cleaning duration.	Should

3.9 Authentication, Staff and Admin (M9, M10)

ID	Requirement	Priority
FR-AU-01	Staff shall log in with PIN (floor roles) or email and password (manager, owner).	Must

ID	Requirement	Priority
FR-AU-02	Access shall be enforced by role as per the permission matrix in SRMS-02.	Must
FR-AU-03	Owner shall manage staff, roles, menu, tables, taxes, thresholds and gateway keys.	Must
FR-AU-04	Manager shall manage shifts and table assignment.	Should
FR-AU-05	Sensitive actions shall be audit logged.	Must
FR-AU-06	Menu shall be importable by CSV.	Could

3.10 Reports (M12) and Feedback/Waitlist (M13)

ID	Requirement	Priority
FR-RP-01	Reports shall cover sales by day/hour/category/item and by payment method.	Must
FR-RP-02	Reports shall cover table turnover, dwell time, cleaning time, alert response time and prep time.	Should
FR-RP-03	Reports shall export to CSV and PDF.	Should
FR-FB-01	Guests shall be able to rate their experience after payment; low ratings alert the manager.	Should
FR-WL-01	Staff shall manage a waiting list and notify guests by SMS or WhatsApp when a table is ready.	Could
FR-WL-02	Staff shall create reservations that mark tables Reserved.	Could

3.11 Phase 2: AI Camera Detection (M14)

ID	Requirement	Priority
FR-AI-01	The system shall map camera views to table zones.	Must (Phase 2)
FR-AI-02	The AI service shall detect people and report seated, left and cleared events with confidence scores.	Must (Phase 2)
FR-AI-03	Events above the configured confidence shall update the state engine; others shall appear as staff suggestions.	Must (Phase 2)
FR-AI-04	The system shall estimate entrance queue size and guest wait time.	Should (Phase 2)
FR-AI-05	The system shall flag tables with guests waiting for staff beyond a threshold.	Should (Phase 2)
FR-AI-06	The system shall not perform face recognition or store identifying images.	Must (Phase 2)
FR-AI-07	Manual overrides shall always take priority over AI events.	Must (Phase 2)

4. External Interface Requirements

Interface	Requirement
User interface	Mobile-first guest PWA; responsive dashboards; kitchen display optimised for distance reading; accessible colours plus icons (not colour alone)
Payment gateway	REST API and signed webhooks (Razorpay or equivalent); UPI, cards, wallets
Messaging	SMS/WhatsApp provider for receipts and waitlist; web push for staff
Printers (optional)	Kitchen or bill printer through browser print or network printer integration
Cameras (Phase 2)	RTSP streams processed by an edge or cloud AI service that posts signed events to the API

5. Non-Functional Requirements

Category	Requirement
Performance	Menu loads in under 2 seconds on 4G; floor map updates within 2 seconds of an event; API p95 under 300 ms for standard calls
Capacity	Support at least 100 tables and 500 concurrent guest sessions per outlet without degradation

Category	Requirement
Availability	99.5% monthly uptime target; graceful degradation: ordering continues, cash and manual flows remain available
Reliability	No lost orders; orders saved before acknowledgement; automatic reconnect and state resync for real-time clients
Security	HTTPS/TLS, hashed credentials, role checks on every request, rate limiting, input validation, protection against QR token guessing, encrypted secrets, OWASP Top 10 controls
Payments	No card data stored; webhook signature verification; idempotency; reconciliation report
Privacy	Compliance with India DPDP Act principles; minimal personal data; AI phase stores no faces
Usability	Guest can order within 3 taps from scan to cart; staff actions in one tap; large touch targets; English plus Hindi/Gujarati
Compatibility	Latest two versions of major mobile and desktop browsers
Maintainability	Modular codebase, automated tests, CI/CD, documented API, structured logging and monitoring
Scalability	Stateless API behind a load balancer; Redis for pub/sub; multi-outlet via restaurant_id
Auditability	Immutable table status log and audit log

6. Use Cases

UC-1 Guest orders and pays

- **Actor:** Customer. **Pre:** table has a QR code. **Post:** bill paid, table flagged for cleaning.
- **Main flow:** scan QR, see menu, add items, place order, track status, request bill, pay online, receive receipt.
- **Alternate:** guest chooses cash and the waiter records it. **Exception:** payment fails, so a Payment Failed alert is raised and the guest may retry or pay cash.

UC-2 Manager monitors the floor

- **Actor:** Manager. **Main flow:** open dashboard, see colours and timers, click a red-flagged table, view alert, assign or resolve, optionally override status.
- **Post:** alert resolved and logged.

UC-3 Cleaner resets a table

- **Actor:** Cleaner. **Pre:** table is Needs Cleaning. **Main flow:** open queue, start task, clean, tap Done.
- **Post:** table is Available and visible as green to the manager.

UC-4 Guest calls the waiter

- **Actor:** Customer. **Main flow:** tap Call Waiter, alert appears on the waiter phone and the table shows Needs Attention, waiter acknowledges and resolves.
- **Exception:** no acknowledgement within the threshold, so the alert escalates to the manager.

7. Acceptance Criteria (Phase 1 MVP)

- A guest can scan, order, and pay online without installing an app.
- The manager sees the table turn Occupied within 2 seconds of a scan.
- A Call Waiter tap shows Needs Attention on the map and on the waiter device within 2 seconds.
- Successful payment closes the session and moves the table to Needs Cleaning.
- Cleaner Done moves the table to Available and logs the duration.
- Prices on the bill match the menu with correct GST; invoice numbers are sequential.
- Duplicate payment attempts do not charge twice.
- All table state changes appear in the status log.

8. Traceability to Modules

Requirement group	Module	Document
FR-QR	M1	SRMS-03, SRMS-06
FR-MN	M2	SRMS-03, SRMS-08
FR-OR / FR-KT	M3, M4	SRMS-03, SRMS-06
FR-PY	M5	SRMS-04, SRMS-06
FR-TB / FR-AL / FR-CL	M6, M7, M8	SRMS-06, SRMS-09
FR-AU	M9, M10	SRMS-02, SRMS-07
FR-AI	M14	SRMS-10

SMART RESTAURANT MANAGEMENT SYSTEM

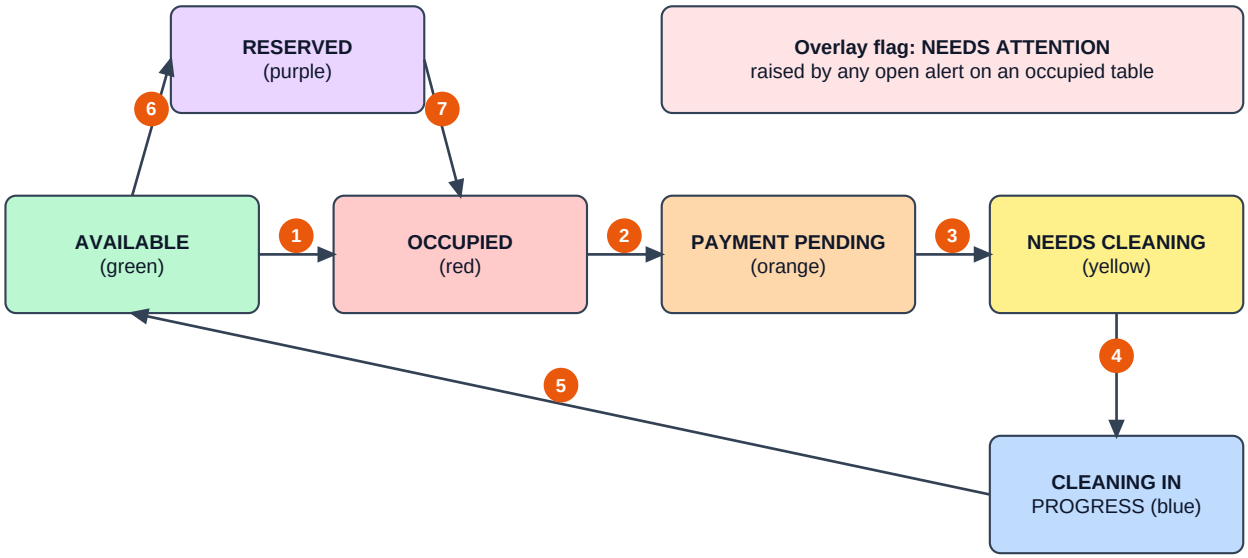
Workflows and State Machines

Table states, order lifecycle, alert rules and process flows

DOCUMENT ID	SRMS-06
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	The behavioural heart of the system: how table status changes, when alerts fire, and how each process flows.

1. Table State Machine

Table status is owned by the **state engine** in the backend. No client can write a status directly; clients send events (scan, order placed, payment received, cleaning done) and the engine decides the transition. Every transition is logged.



#	Transition	Trigger event	Guard / side effect
1	Available to Occupied	Guest scans QR, or staff seats a walk-in, or Phase 2 detects guests seated	Creates table session; starts timer
2	Occupied to Payment Pending	Guest requests bill or staff generates bill	Locks bill total; raises Request Bill alert
3	Payment Pending to Needs Cleaning	Bill fully paid (gateway webhook or cash)	Closes session; creates cleaning task
4	Needs Cleaning to Cleaning In Progress	Cleaner taps Start	Records start time
5	Cleaning In Progress to Available	Cleaner taps Done	Records duration; clears flags; checks waitlist
6	Available to Reserved	Reservation window starts (for example 15 minutes before)	Cleaning priority increases before this
7	Reserved to Occupied	Reserved guest arrives and is seated	Links reservation to session

Other transitions: any state to **Out of Service** by cleaner/manager with a note, and back to Needs Cleaning when restored. Manager override can set any valid state with a mandatory reason. Occupied to Needs Cleaning is allowed if guests walk out (close reason WALK_OUT) and the manager closes the session.

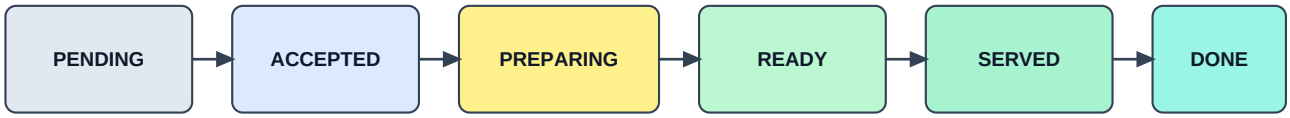
2. Floor Map Colour and Icon Legend

State / flag	Colour	Icon / extra info on tile
Available	Green	Seats count
Occupied	Red	Guest count, session timer
Payment Pending	Orange	Bill amount, payment icon
Needs Cleaning	Yellow	Time waiting for cleaner
Cleaning In Progress	Blue	Cleaner name, timer

State / flag	Colour	Icon / extra info on tile
Reserved	Purple	Reservation time and name
Out of Service	Grey	Reason
Needs Attention (overlay)	Pulsing ring or bell icon on any tile	Alert type, age of alert

Accessibility: colour is never the only signal. Each state also has a text label and icon so colour-blind staff can read the map.

3. Order and Item Lifecycle



- Order status is derived from item statuses (for example Ready when all active items are Ready).
- Cancellation: Pending and Accepted items can be cancelled by guest or staff; Preparing or later needs manager approval and records waste.
- An item marked out of stock by the kitchen is cancelled with reason and the guest is notified with an option to replace it.

4. Automatic Alert Rules

All thresholds are defaults and editable in admin settings.

Alert	Condition (default)	First recipient	Escalation
Call Waiter	Guest taps Call Waiter	Assigned waiter	Manager after 90 seconds unacknowledged
Request Bill	Guest taps Request Bill	Assigned waiter	Manager after 120 seconds
Long Wait (no order)	Table Occupied for 10 minutes with no order	Assigned waiter	Manager after 5 more minutes
Order Delayed	Any item exceeds its prep time by 5 minutes	Kitchen and waiter	Manager after 10 minutes
Unattended Table	Orders served but no staff interaction for 25 minutes	Assigned waiter	Manager
Payment Failed	Gateway reports failure	Assigned waiter	None; guest may retry or pay cash
Long Stay	Session longer than 90 minutes while waitlist is not empty	Manager	None
Cleaning Delay	Needs Cleaning for more than 5 minutes	Cleaners and manager	Manager after 10 minutes
AI Suggestion (Phase 2)	Camera detects table cleared or guests waiting	Waiter or manager	Confirm or dismiss

5. Key Process Flows

5.1 Guest scans and orders

1. Guest scans QR; browser opens the PWA with the table token.
2. Backend validates the token, finds or creates the active session, issues a session token, sets table Occupied (event logged).

3. Guest fills the shared cart; the server prices it; guest taps Place Order.
4. Backend saves the order and items in one transaction, then publishes events: kitchen ticket, waiter notification, floor map update.
5. Kitchen updates item status; events push live status to the guest and the map.

5.2 Payment

1. Guest taps Pay; backend creates a payment record with an idempotency key and a gateway order.
2. Guest completes UPI or card payment in the gateway sheet.
3. Gateway calls the webhook; backend verifies the signature and marks the payment SUCCESS.
4. If the bill is fully paid: session closes, table moves to Needs Cleaning, cleaning task is created, invoice is issued.
5. Guest receives a receipt. The client never decides success on its own; it polls or listens for the server confirmation.

5.3 Cleaning

1. Cleaning task appears in the cleaner queue with a push sound.
2. Cleaner taps Start (table turns blue), cleans, taps Done.
3. Table becomes Available (green). The waitlist is checked and the next party is notified if any.

5.4 Alert handling

1. Alert is raised (guest, system rule or AI) and the table gets the Needs Attention flag.
2. Assigned waiter is notified. Acknowledge shows who is handling it.
3. Waiter resolves after serving the guest; flag clears when no open alerts remain.
4. If unacknowledged past the threshold, the alert escalates to the manager and is highlighted.

6. Edge Cases and Rules

Case	Handling
Guest scans the wrong table QR	Show table label prominently at the top with a Change table option; staff can move the session
Several guests scan the same table	All join the same session and cart; each gets the same live status
Guest scans after paying and leaving	Session closed, so a new session is started only if the table is Available or staff reopen it; prevents ordering to a dirty table
Guest leaves without paying	Manager closes session with reason WALK_OUT; bill stays unpaid and is reported; table goes to Needs Cleaning
Payment succeeded but guest closed the browser	Webhook still completes payment and closes the bill
Duplicate webhook or double tap	Idempotency keys and unique provider payment IDs ignore duplicates
Internet drops in kitchen	Local retry with banner; tickets resync on reconnect; waiter can announce by voice; no orders lost
Item price changed during session	Order uses the price at order time (snapshot); new orders use the new price
Table moved or merged for a big group	Manager merges sessions; floor map shows linked tables
Manager override conflicts with AI event (Phase 2)	Manual override wins and AI events are ignored for a cooldown period

SMART RESTAURANT MANAGEMENT SYSTEM

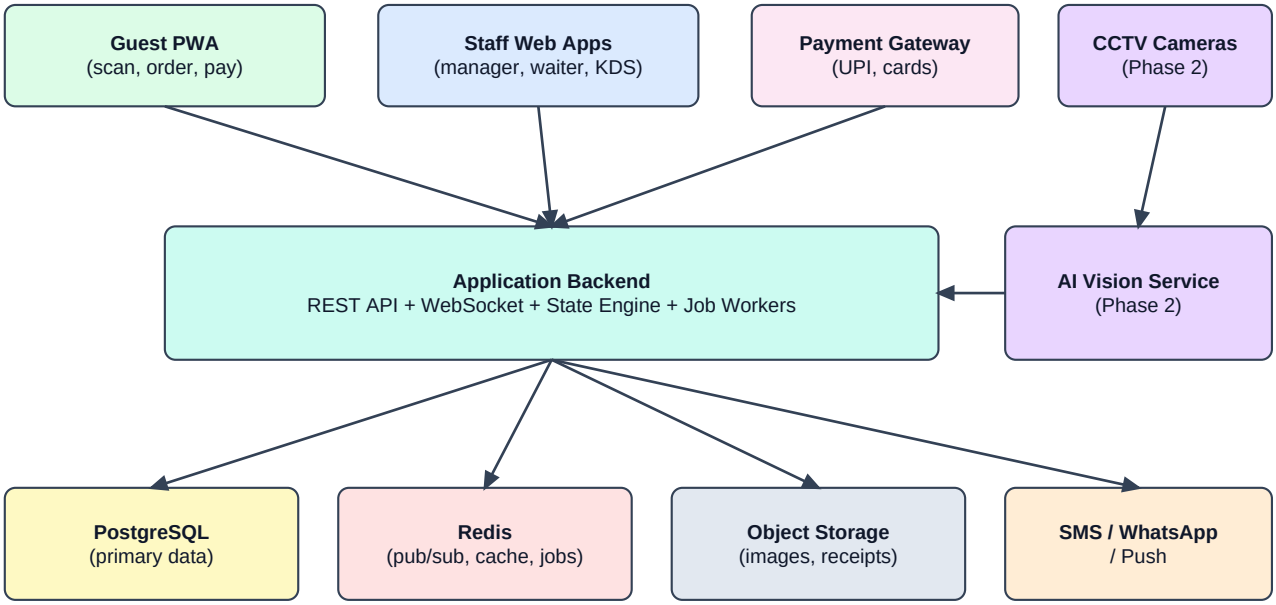
System Architecture and Tech Stack

Components, technologies, real-time, security and deployment

DOCUMENT ID	SRMS-07
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	How the system is built: components, recommended stack, real-time design, security and the Phase 2 AI pipeline.

1. Architecture Overview

A modular monolith is recommended for Phase 1: one deployable backend with clear internal modules, a single PostgreSQL database and Redis for real-time messaging. This is simple to build and operate, and the AI vision service is the only separate service, added in Phase 2.



2. Recommended Technology Stack

Layer	Choice	Why
Guest app	Next.js (React) PWA, Tailwind CSS	No install needed; fast on mobile; SEO and QR deep links work well
Staff apps	React (same codebase or separate routes), installable PWA	One skill set; works on tablets and phones; push notifications
Backend	Node.js with NestJS (TypeScript)	Modular structure matches our modules; strong typing; good WebSocket support
Real-time	Socket.IO over Redis adapter	Live floor map, kitchen tickets, alerts; auto-reconnect; rooms per restaurant and per table
Database	PostgreSQL 15+ with Prisma or TypeORM	Transactions for orders and payments; partial unique indexes; JSONB
Cache / queues	Redis with BullMQ	Timers for alert rules, retries, webhooks processing
Payments	Razorpay (UPI, cards, wallets) with webhook verification	India focus; UPI first; good docs and test mode
Notifications	Web Push, WhatsApp Business or SMS provider	Receipts, waitlist messages, staff alerts
Storage	S3-compatible bucket plus CDN	Menu images, QR PDFs, receipts
Auth	JWT access and refresh tokens, RBAC guard, argon2 hashing	Stateless scaling; role checks on every route
Hosting	Docker on a managed cloud (AWS, GCP or DigitalOcean), managed Postgres	Simple operations; backups handled
CI/CD and quality	GitHub Actions, ESLint, Jest, Playwright, Sentry	Automated testing and error tracking

Layer	Choice	Why
AI (Phase 2)	Python service, YOLO-family detector via ONNX/OpenCV, edge device optional	Real-time person/object detection; can run on a small GPU or edge box

3. Backend Module Layout

```
backend/src
  modules/
    session/      (QR, table sessions)
    menu/         (categories, items, modifiers)
    orders/       (cart, orders, items)
    kitchen/      (KDS queries, item status)
    billing/      (bills, tax, invoices)
    payments/     (gateway, webhooks, refunds)
    tables/       (tables, floors, state engine)
    alerts/       (rules, escalation)
    cleaning/     (tasks, queue)
    auth/         (staff, roles, guards)
    admin/        (settings, QR generation)
    reports/      (analytics queries)
    vision/       (Phase 2 event ingestion)
  realtime/       (socket gateway, event publisher)
  common/         (guards, filters, audit, utils)
  jobs/           (timers, schedulers)
```

4. Real-time Design

- Domain events (TableOccupied, OrderPlaced, ItemReady, AlertRaised, PaymentSucceeded, TableCleaned) are published after each database commit.
- Staff clients join a **restaurant room**; guest clients join a **session room** and only receive their own table events.
- On reconnect, clients fetch a fresh snapshot (floor map, open alerts, active orders) and then continue with live events, so nothing is missed.
- Redis pub/sub lets multiple backend instances broadcast consistently.
- Timers (idle sessions, delayed orders, unattended tables) run as scheduled jobs that check state and emit alert events.

5. State Engine Design

- A single module is the only code allowed to update `tables.status`.
- Input: events with actor and reason. The engine validates the transition against the allowed-transition table, writes the new status and the log row in one transaction, then publishes an event.
- Row-level locking on the table row avoids race conditions (two scans at once, payment and override at once).
- Phase 2 AI events use exactly the same input path with `actor_type = AI` and a confidence value.

6. Security Architecture

Area	Control
Transport	HTTPS only, HSTS, secure WebSocket
QR abuse	Long random table tokens, rate limiting per IP and token, session token binding, rotating QR if abused
Authentication	Hashed passwords and PINs, lockout, refresh token rotation, optional 2FA for manager/owner
Authorisation	Role guards on every endpoint; restaurant_id enforced in every query (tenant isolation)
Input	Server-side validation, parameterised queries, output encoding, file type and size limits
Payments	Server-calculated amounts, signed webhooks, idempotency keys, no card data stored

Area	Control
Secrets	Environment secrets in a vault or encrypted store; gateway keys encrypted at rest
Data protection	Encrypted backups, least-privilege database users, audit log
Camera data (Phase 2)	Encrypted stream URLs, edge processing, no face data, short snapshot retention

7. Deployment and Environments

Environment	Purpose	Notes
Local	Developer machines	Docker Compose: API, Postgres, Redis
Staging	Testing and demos	Gateway in test mode; seeded demo restaurant
Production	Live restaurants	Managed Postgres with backups, autoscaling API, monitoring and alerts

- CI pipeline: lint, unit tests, integration tests, build image, deploy to staging, smoke test, manual approval to production.
- Observability: structured logs, error tracking (Sentry), metrics and uptime checks, alert if payment webhook failures spike.
- Backups: daily full plus point-in-time recovery; restore drill every quarter.

8. Phase 2 Architecture: AI Camera Pipeline

1. Cameras stream RTSP video to an edge box in the restaurant (or a cloud GPU service).
2. The vision service samples frames (for example 2 to 5 per second), runs person/chair/table detection and counts people per table zone.
3. A small rules layer turns raw detections into events such as PERSON_SEATED, TABLE_CLEARED, LONG_UNATTENDED, QUEUE_COUNT with a confidence score.
4. Events are posted to the backend vision endpoint with a signed token.
5. The state engine applies high-confidence events and creates suggestions for low-confidence ones; manual actions always win.
6. Only counts, states and optional short-lived snapshots leave the edge device.

SMART RESTAURANT MANAGEMENT SYSTEM

API Specification

REST endpoints and real-time events

DOCUMENT ID	SRMS-08
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	Contract between the clients and the backend: endpoints, roles, events and sample payloads.

1. Conventions

- Base URL: `/api/v1`. JSON request and response. UTF-8. Timestamps in ISO 8601 UTC.
- Auth types: **Guest** (session token from QR scan), **Staff** (JWT with role), **Service** (signed token for AI service and payment webhooks), **Public** (none).
- Errors use a standard body: `{ "error": { "code": "TABLE_NOT_AVAILABLE", "message": "..." } }` with proper HTTP status.
- Lists are paginated with `?page=&limit=`. State-changing payment endpoints require an `Idempotency-Key` header.
- All amounts are in paise or fixed decimals as strings to avoid rounding errors (final choice recorded in the API contract).

2. REST Endpoints

2.1 Session and Menu (Guest)

Method	Path	Auth	Description
POST	/sessions/scan	Public	Body: table token. Creates or joins the active session; returns session token and table info
GET	/sessions/current	Guest	Current session, table label, guest count
PATCH	/sessions/current	Guest	Set guest count or phone
GET	/menu	Guest	Categories, items, variants, modifiers, availability
GET	/menu/items/:id	Guest	Item detail

2.2 Cart, Orders and Alerts (Guest)

Method	Path	Auth	Description
GET	/cart	Guest	Shared cart for the session
PUT	/cart/items	Guest	Add, update or remove item with variant, modifiers, note
POST	/orders	Guest	Place order from the cart; server recalculates prices
GET	/orders	Guest	Orders and live item status for the session
POST	/orders/items/:id/cancel	Guest	Cancel if still Pending or Accepted
POST	/alerts/call-waiter	Guest	Raise Call Waiter alert
POST	/alerts/request-bill	Guest	Raise Request Bill alert and set Payment Pending

2.3 Billing and Payments

Method	Path	Auth	Description
GET	/bill	Guest	Running bill with tax breakdown
POST	/payments	Guest	Create payment (method, amount); returns gateway order. Needs Idempotency-Key
GET	/payments/:id	Guest	Payment status (server truth)
POST	/webhooks/razorpay	Service	Gateway webhook; signature verified
POST	/payments/cash	Staff (waiter+)	Record cash payment for the bill
POST	/payments/:id/refund	Manager+	Refund with reason; owner above limit
POST	/bills/:id/discount	Manager+	Apply discount with reason (audited)
GET	/bills/:id/invoice	Guest / Staff	PDF invoice or receipt link

2.4 Staff: Tables, Floor Map and Sessions

Method	Path	Auth	Description
GET	/floor	Staff	Snapshot: floors, tables, status, guests, timers, open alert counts
GET	/tables/:id	Staff	Table detail with session, orders, bill, alerts
POST	/tables/:id/seat	Waiter+	Seat walk-in; create session with guest count
POST	/tables/:id/status	Manager+	Override status; body: status, reason
POST	/sessions/:id/close	Manager+	Close session with reason (PAID, WALK_OUT, MANAGER)
GET	/tables/:id/history	Manager+	Status log
GET	/dashboard/summary	Manager+	Occupancy, capacity %, average dwell, open alerts

2.5 Staff: Orders and Kitchen

Method	Path	Auth	Description
GET	/kitchen/tickets	Kitchen+	Open items ordered by time
PATCH	/order-items/:id/status	Kitchen / Waiter	Accept, Preparing, Ready (kitchen); Served (waiter)
POST	/staff/orders	Waiter+	Place order on behalf of a table
PATCH	/order-items/:id	Waiter+	Edit quantity or note; cancel with reason
PATCH	/menu/items/:id/availability	Kitchen+	Mark sold out or available

2.6 Alerts and Cleaning

Method	Path	Auth	Description
GET	/alerts	Staff	Open alerts (filter by table, type, assignee)
POST	/alerts/:id/acknowledge	Waiter+	Acknowledge
POST	/alerts/:id/resolve	Waiter+	Resolve
GET	/cleaning/tasks	Cleaner+	Queue ordered by priority and age
POST	/cleaning/tasks/:id/start	Cleaner+	Start; table becomes Cleaning In Progress
POST	/cleaning/tasks/:id/done	Cleaner+	Complete; table becomes Available
POST	/tables/:id/out-of-service	Cleaner+	Mark out of service with note

2.7 Admin

Method	Path	Auth	Description
POST/PUT/DELETE	/admin/menu/categories, /admin/menu/items	Manager+	Menu CRUD, image upload, CSV import
POST/PUT	/admin/floors, /admin/tables	Owner	Floors, tables and layout positions
POST	/admin/tables/:id/qr/rotate	Owner	Rotate QR token
GET	/admin/qr/sheet	Owner	Printable PDF of all table QR codes
CRUD	/admin/staff	Owner	Staff accounts and roles
PUT	/admin/settings	Owner	Taxes, thresholds, timeouts, gateway keys
POST	/auth/login, /auth/pin-login, /auth/refresh	Public	Staff authentication

2.8 Reports

Method	Path	Auth	Description
GET	/reports/sales	Manager+	Sales by period, category, item, payment method
GET	/reports/tables	Manager+	Turnover, dwell, cleaning time
GET	/reports/service	Manager+	Alert response, prep time
GET	/reports/export	Manager+	CSV or PDF export

2.9 Phase 2: Vision

Method	Path	Auth	Description
POST	/vision/events	Service	Batch of detection events with confidence, camera, zone
GET	/vision/suggestions	Manager+	Pending low-confidence suggestions
POST	/vision/suggestions/:id/confirm	Waiter+	Apply suggestion via state engine
POST	/vision/suggestions/:id/dismiss	Waiter+	Dismiss suggestion
CRUD	/admin/cameras, /admin/cameras/:id/zones	Owner	Cameras and table zones

3. WebSocket Events

Namespace `/live`. Staff join `restaurant:{id}`; guests join `session:{id}`.

Event	Audience	Payload (summary)
table.updated	Staff	tableId, status, guestCount, statusChangedAt, alertCount
session.opened / session.closed	Staff	sessionId, tableId, reason
order.placed	Kitchen, Waiter, Manager	orderId, tableLabel, items
item.status	Guest, Staff	orderItemId, status, timestamps
alert.raised / alert.updated	Staff	alertId, tableId, type, severity, status, assignee
cleaning.task	Cleaner, Manager	taskId, tableId, status, priority
bill.updated	Guest, Waiter	total, amountPaid, status
payment.status	Guest, Waiter	paymentId, status
menu.availability	Guest, Staff	itemId, isAvailable
cart.updated	Guest	cart snapshot
vision.suggestion (Phase 2)	Manager, Waiter	suggestionId, tableId, type, confidence

4. Example Payloads

```
POST /api/v1/sessions/scan
{ "tableToken": "k9X2..." }

200 OK
{ "sessionToken": "eyJ...",
  "table": { "label": "T04", "floor": "Indoor" },
  "session": { "id": "...", "startedAt": "2026-11-06T07:10:00Z" } }
```

```
POST /api/v1/tables/:id/status   (Manager)
{ "status": "AVAILABLE", "reason": "Guests left, table already clean" }

200 OK
{ "tableId": "...", "from": "NEEDS_CLEANING", "to": "AVAILABLE" }
```

SMART RESTAURANT MANAGEMENT SYSTEM

UI/UX Screen Specification

Screens, wireframe and interaction design

DOCUMENT ID	SRMS-09
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	What each user sees and does, including the manager floor map wireframe.

1. Design Principles

- **Guest:** mobile first, one hand, large touch targets, under three taps from scan to cart, works on slow 4G.
- **Manager:** glanceable. The floor map must be understood in two seconds from across the counter.
- **Staff:** one tap per action, big buttons, sound and vibration for important events, minimal typing.
- **Kitchen:** high contrast, large type readable from a distance, colour timers.
- **Accessibility:** colour plus icon plus text for every status; support for larger fonts; English with Hindi and Gujarati.

2. Sitemap

Interface	Screens
Guest PWA	Welcome and table confirmation; Menu; Item detail with options; Cart; Order status; Bill and payment; Receipt and feedback
Manager dashboard	Live floor map; Table detail panel; Alerts feed; Orders overview; Waitlist and reservations; Reports; Shift and staff view
Waiter app	My tables; Table detail; New order for table; Alerts; Cash payment
Kitchen display	Ticket board; Sold-out toggles
Cleaner app	Cleaning queue; Task detail; Out of service
Admin panel	Menu management; Floor and table designer; QR sheet; Staff; Settings; Audit log; Cameras (Phase 2)

3. Manager Live Floor Map (Key Screen)

This wireframe shows the main idea: coloured table tiles, an alert badge on tables needing attention, KPI cards, an alert list and a detail panel.



- **Tile:** table label, state colour, guest count and session timer, or cleaning wait time. An orange badge shows Needs Attention.
- **Click a tile:** opens the detail panel with orders, bill, alerts and actions (seat, override, close).
- **Filters:** by floor, by waiter, by state. A toggle to show only tiles that need action.
- **Sounds and highlight:** new alerts pulse the tile and play a soft sound; unacknowledged alerts turn red after the escalation time.
- **Edit mode (Owner):** drag tiles to match the real layout.

4. Guest Screens

Screen	Content and behaviour
Welcome	Restaurant logo, big table label (T04) with Change table link, optional guest count; Continue button
Menu	Sticky category tabs, search, veg toggle, item cards with image, price and Add button; floating cart bar with count and total; sold-out items greyed
Item detail	Photo, description, variants, modifier groups with validation, note field, quantity stepper, Add to cart with price
Cart	Shared with table guests (avatars or labels), edit quantities, order note, price breakdown, Place Order
Order status	Per-item timeline Placed, Preparing, Ready, Served; estimated time; Add more items button
Help bar	Always visible: Call Waiter and Request Bill buttons with confirmation toast
Bill and payment	Itemised bill, GST, service charge toggle, tip chips, split option, Pay with UPI/card, Pay cash at table
Receipt and feedback	Payment success, invoice download, star rating and comment

5. Staff Screens

Screen	Content and behaviour
Waiter: My tables	List and mini-map of assigned tables with state colours; red bell on tables needing attention; tap for detail
Waiter: Alerts	Cards with table, type, age; Acknowledge then Resolve buttons; sound and vibration
Waiter: New order	Table selector, same menu UI optimised for fast entry, send to kitchen
Kitchen: Ticket board	Columns New, Preparing, Ready; ticket shows table, items, notes, timer; one-tap status buttons; sold-out toggles
Cleaner: Queue	Sorted cards: table label, waiting time, priority tag (reserved soon); Start and Done big buttons; Out of Service with reason
Admin: Table designer	Canvas with drag-and-drop tables, seats and shape; QR sheet generation; floor tabs

6. Notification Design

Event	Who	How
New order	Kitchen, waiter	Ticket appears, short chime
Call waiter / request bill	Waiter, manager	Push, vibration, bell badge on tile
Escalated alert	Manager	Red tile pulse, louder tone
Table needs cleaning	Cleaner	Push, queue card on top
Item ready	Waiter, guest	Waiter push; guest status updates
Payment success	Guest, waiter	Receipt screen; table turns yellow on map

7. Visual Style

Element	Guideline
Palette	Neutral base with brand teal; status colours fixed as in the legend (green, red, orange, yellow, blue, purple, grey)
Typography	Clean sans-serif (Inter or system font); minimum 16 px body on guest app; large numerals for timers
Components	Rounded cards, large buttons, clear focus states; skeleton loaders; offline banner
Motion	Subtle: pulse for alerts, slide for cart; respect reduced-motion setting

Element	Guideline
Dark mode	Supported on staff dashboard and kitchen display (useful in dim kitchens)

8. Phase 2 UI Additions

- AI badge on tiles when status came from a camera; click to see event time and confidence.
- Suggestions tray: for example Table T07 appears cleared. Confirm or Dismiss.
- Entrance queue card with live people count and estimated wait time.
- Camera health panel in admin: online status and zone editor over a still frame.

SMART RESTAURANT MANAGEMENT SYSTEM

Roadmap, Testing and AI Phase Plan

Timeline, team, quality, risks and Phase 2 rollout

DOCUMENT ID	SRMS-10
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	How and when we will deliver, how we will test it, what could go wrong, and how AI camera detection is introduced safely.

1. Delivery Roadmap

Estimates assume a small team (see section 2) and are indicative; they will be refined after design sign-off.

Phase	Weeks	Deliverables	Exit criteria
0. Discovery and design	1 to 2	Approved documents, UI wireframes and design system, API contract, test restaurant details	Sign-off on SRS and UX
1A. Foundation	3 to 5	Auth and roles, restaurants/floors/tables, QR generation, menu admin, CI/CD, staging	Staff can log in and set up a restaurant
1B. Ordering	6 to 9	QR session, menu, shared cart, orders, kitchen display, real-time events	Guest order reaches kitchen in under 2 seconds
1C. Payments and billing	10 to 12	Bill, GST, UPI/card via gateway, cash, receipts, refunds	Payments reconcile in test and live sandbox
1D. Floor control	13 to 16	State engine, live floor map, alerts engine, cleaning queue, basic reports	Full table lifecycle works end to end
1E. Hardening and pilot	17 to 20	Load, security and usability testing, fixes, pilot in one restaurant	Pilot KPIs met; go-live decision
1.5 Enhancements	21 to 24	Waitlist, reservations, feedback, split bill, multi-language	Adoption by pilot staff
2. AI camera detection	25 to 36	Camera zones, detection service, suggestions, auto-update, queue and dwell analytics	Accuracy targets met in pilot
3. Scale	37+	Multi-outlet, loyalty, advanced analytics, native apps if needed	Business decision

2. Suggested Team

Role	Count	Notes
Product owner	1	You; decisions on scope and priorities
UI/UX designer	1	Part-time after Phase 0
Full-stack developers (Node, React)	2 to 3	Core build
QA engineer	1	Automation and manual testing
DevOps (part-time)	1	CI/CD, hosting, monitoring
ML engineer (Phase 2)	1	Vision model, edge deployment

3. Testing Strategy

Level	Scope	Tools
Unit tests	State engine transitions, price and tax calculation, alert rules, permissions	Jest
Integration tests	API with real database: session, order, payment, webhook flows	Jest + Supertest, test containers
End-to-end tests	Guest scan to pay; manager sees states; cleaner resets table	Playwright (mobile and desktop viewports)
Concurrency tests	Two scans at once, duplicate webhooks, payment plus override race	Custom scripts, k6
Load tests	500 concurrent guest sessions and 100 staff sockets per outlet	k6
Security tests	OWASP Top 10 checks, QR token guessing, role bypass, webhook forgery	ZAP, manual review
Usability tests	Real waiters, kitchen and cleaners in pilot; measure taps and errors	Observation and surveys

Level	Scope	Tools
Device tests	Low-end Android phones, iPhone Safari, kitchen tablet	BrowserStack plus real devices

Critical test scenarios

- Table state follows the full cycle Available, Occupied, Payment Pending, Needs Cleaning, Cleaning, Available with correct log entries.
- Payment webhook arrives twice: only one success and one cleaning task.
- Guest closes the browser after paying: table still moves correctly.
- Network loss on the manager screen: after reconnect the map matches the server.
- Unacknowledged alert escalates at the configured time.
- Role tests: cleaner cannot see payments; waiter cannot refund.

4. Phase 2 Plan: AI Camera Detection

4.1 What we will detect

Capability	Output event	Use
People per table zone	PERSON_SEATED / PERSON_LEFT	Auto-occupy and auto-release suggestions; guest count
Table cleared	TABLE_CLEARED	Suggest Needs Cleaning to Available when plates are removed and the cleaner has wiped
Staff near table	STAFF_VISIT	Reset unattended timers
Guest waiting for staff	LONG_UNATTENDED	Raise Needs Attention
Entrance crowd	QUEUE_COUNT	Estimate wait time and feed waitlist
Dwell time	Computed from events	Analytics and long-stay alerts

4.2 Rollout steps

1. Install or reuse existing CCTV; verify camera angles cover tables without privacy-sensitive areas.
2. Calibrate: draw table zones on a still frame for each camera.
3. **Shadow mode:** the AI only logs events, and we compare to real table status for 2 to 4 weeks.
4. **Suggestion mode:** staff see suggestions and confirm or dismiss; measure precision and recall.
5. **Auto mode:** high-confidence events update status automatically; manual override always wins.
6. Continuous tuning using confirmed and dismissed suggestions.

4.3 Targets and safeguards

- Targets for auto mode: at least 95% precision on occupied/free detection; clear fallback when cameras are offline.
- Privacy: no face recognition; process on the edge; store only counts and states; snapshots optional with short retention; visible notice to guests and staff as required by law and policy.
- Reliability: heartbeats from each camera; if a camera is offline, tables in its zone fall back to event-driven logic automatically.

5. Risks and Mitigations

Risk	Impact	Mitigation
Staff forget to tap Cleaned or Served	Wrong map status	Timers and reminders; gamified cleaning times; Phase 2 AI confirms
Guests do not scan or prefer waiter ordering	Lower adoption	Waiter order screen; table tent cards; small incentive; simple UI

Risk	Impact	Mitigation
Poor restaurant internet	Missed updates	Resync on reconnect; offline banners; backup mobile hotspot recommendation
Payment edge cases (webhook delay, duplicates)	Financial errors	Idempotency, webhook-first confirmation, daily reconciliation
QR misuse (photo of QR, remote ordering)	Fake orders	Session binding, rate limits, optional waiter confirmation, location hint or short code on table
Scope creep	Delay	Strict phase gates; backlog prioritisation by product owner
Camera accuracy and privacy concerns (Phase 2)	Wrong states, legal risk	Shadow mode, confidence thresholds, no biometric data, clear signage
Staff resistance to change	Poor usage	Training, pilot champions, simple one-tap actions

6. Open Decisions

Decision	Options	Recommendation
Payment gateway	Razorpay, Cashfree, PayU	Razorpay for UPI-first and docs; confirm fees
Guest identity	Anonymous QR only, or phone OTP	Anonymous first; optional phone for receipt and waitlist
Waiter confirmation of QR orders	Auto-send to kitchen, or waiter confirms	Configurable; default auto-send
Language support at launch	English only, or plus Hindi and Gujarati	English at launch, add Hindi and Gujarati in 1.5
Hosting region	India region of chosen cloud	India region for latency and data residency
Number of outlets at launch	One pilot outlet	One, with multi-tenant schema from day one

7. Next Steps

1. Review and approve this document set (overview, roles, modules, schema, SRS, workflows, architecture, API, UI/UX).
2. Confirm open decisions in section 6.
3. Produce clickable UI prototype for the guest app and the manager floor map.
4. Set up repository, CI/CD and staging, then start Phase 1A.